

# First Order Logic

- in propositional logic, we can only represent the facts, which are either true or false. PL is not sufficient to represent the complex sentences
- The propositional logic has very limited expressive power. Consider the following sentence, which we cannot represent using PL logic.
- **"Some humans are intelligent", or**
- **"Sachin likes cricket."**

# First-order logic

- First-order logic (FOL) models the world in terms of
  - **Objects**, which are things with individual identities
  - **Properties** of objects that distinguish them from other objects
  - **Relations** that hold among sets of objects
  - **Functions**, which are a subset of relations where there is only one “value” for any given “input”
- Examples:
  - Objects: Students, lectures, companies, cars ...
  - Relations: Brother-of, bigger-than, outside, part-of, has-color, occurs-after, owns, visits, precedes, ...
  - Properties: blue, oval, even, large, ...
  - Functions: father-of, best-friend, second-half, one-more-than ...

# User provides

- **Constant symbols**, which represent individuals in the world
  - Mary
  - 3
  - Green
- **Function symbols**, which map individuals to individuals
  - father-of(Mary) = John
  - color-of(Sky) = Blue
- **Predicate symbols**, which map individuals to truth values
  - greater(5,3)
  - green(Grass)
  - color(Grass, Green)

# FOL Provides

- **Variable symbols**

- E.g.,  $x$ ,  $y$ ,  $\text{foo}$

- **Connectives**

- Same as in PL: not ( $\neg$ ), and ( $\wedge$ ), or ( $\vee$ ), implies ( $\rightarrow$ ), if and only if (biconditional  $\leftrightarrow$ )

- **Quantifiers**

- Universal  $\forall x$  or **(Ax)**
- Existential  $\exists x$  or **(Ex)**

# Sentences are built from terms and atoms

- A **term** (denoting a real-world individual) is a constant symbol, a variable symbol, or an  $n$ -place function of  $n$  terms.  
 $x$  and  $f(x_1, \dots, x_n)$  are terms, where each  $x_i$  is a term.  
A term with no variables is a **ground term**
- An **atomic sentence** (which has value true or false) is an  $n$ -place predicate of  $n$  terms
- A **complex sentence** is formed from atomic sentences connected by the logical connectives:  
 $\neg P, P \vee Q, P \wedge Q, P \rightarrow Q, P \leftrightarrow Q$  where  $P$  and  $Q$  are sentences
- A **quantified sentence** adds quantifiers  $\forall$  and  $\exists$
- A **well-formed formula (wff)** is a sentence containing no “free” variables. That is, all variables are “bound” by universal or existential quantifiers.  
 $(\forall x)P(x,y)$  has  $x$  bound as a universally quantified variable, but  $y$  is free.

# Atomic sentences:

- Atomic sentences are the most basic sentences of first-order logic. These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- We can represent atomic sentences as **Predicate (term1, term2, ....., term n)**.
- **Example: Ravi and Ajay are brothers: => Brothers(Ravi, Ajay).**  
**Chinky is a cat: => cat (Chinky).**
-

# Complex Sentences:

- Complex sentences are made by combining atomic sentences using connectives.
- **First-order logic statements can be divided into two parts:**
- **Subject:** Subject is the main part of the statement.
- **Predicate:** A predicate can be defined as a relation, which binds two atoms together in a statement.
- **Consider the statement: "x is an integer."**, it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.
-



# Quantifiers

- **Universal quantification**
  - $(\forall x)P(x)$  means that  $P$  holds for **all** values of  $x$  in the domain associated with that variable
  - E.g.,  $(\forall x) \text{dolphin}(x) \rightarrow \text{mammal}(x)$
- **Existential quantification**
  - $(\exists x)P(x)$  means that  $P$  holds for **some** value of  $x$  in the domain associated with that variable
  - E.g.,  $(\exists x) \text{mammal}(x) \wedge \text{lays-eggs}(x)$
  - Permits one to make a statement about some object without naming it

# Quantifiers

- Universal quantifiers are often used with “implies” to form “rules”:  
 $(\forall x) \text{ student}(x) \rightarrow \text{smart}(x)$  means “All students are smart”
- Universal quantification is *rarely* used to make blanket statements about every individual in the world:  
 $(\forall x) \text{ student}(x) \wedge \text{smart}(x)$  means “Everyone in the world is a student and is smart”
- Existential quantifiers are usually used with “and” to specify a list of properties about an individual:  
 $(\exists x) \text{ student}(x) \wedge \text{smart}(x)$  means “There is a student who is smart”
- A common mistake is to represent this English sentence as the FOL sentence:  
 $(\exists x) \text{ student}(x) \rightarrow \text{smart}(x)$ 
  - But what happens when there is a person who is *not* a student?

# Quantifier Scope

- Switching the order of universal quantifiers *does not* change the meaning:
  - $(\forall x)(\forall y)P(x,y) \leftrightarrow (\forall y)(\forall x) P(x,y)$
- Similarly, you can switch the order of existential quantifiers:
  - $(\exists x)(\exists y)P(x,y) \leftrightarrow (\exists y)(\exists x) P(x,y)$
- Switching the order of universals and existentials *does* change meaning:
  - Everyone likes someone:  $(\forall x)(\exists y) \text{ likes}(x,y)$
  - Someone is liked by everyone:  $(\exists y)(\forall x) \text{ likes}(x,y)$

- Some Examples of FOL using quantifier:

- **1. All birds fly.**

In this question the predicate is "**fly(bird).**"

And since there are all birds who fly so it will be represented as follows.

$$\forall x \text{ bird}(x) \rightarrow \text{fly}(x).$$

- **2. Every man respects his parent.**

In this question, the predicate is "**respect(x, y),**" where **x=man, and y= parent.**

Since there is every man so will use  $\forall$ , and it will be represented as follows:

$$\forall x \text{ man}(x) \rightarrow \text{respects}(x, \text{parent}).$$

- **3. Some boys play cricket.**

In this question, the predicate is "**play(x, y),**" where **x= boys, and y= game.** Since there are some boys so we will use  $\exists$ , and it will be represented as:

$$\exists x \text{ boys}(x) \wedge \text{play}(x, \text{cricket}).$$

-

- **4. Not all students like both Mathematics and Science.**

In this question, the predicate is "**like(x, y),**" where **x= student, and y= subject.** Since there are not all students, so we will use **∀ with negation,** so following representation for this:

$$\neg \forall (x) [ \text{student}(x) \rightarrow \text{like}(x, \text{Mathematics}) \wedge \text{like}(x, \text{Science}) ].$$

- **5. Only one student failed in Mathematics.**

In this question, the predicate is "**failed(x, y),**" where **x= student, and y= subject.** Since there is only one student who failed in Mathematics, so we will use following representation for this:

$$\exists (x) [ \text{student}(x) \rightarrow \text{failed} (x, \text{Mathematics}) \wedge \forall (y) [ \neg (x==y) \wedge \text{student}(y) \rightarrow \neg \text{failed} (x, \text{Mathematics}) ].$$

-

# Free and Bound Variables:

The quantifiers interact with variables which appear in a suitable way. There are two types of variables in First-order logic which are given below:

**Free Variable:** A variable is said to be a free variable in a formula if it occurs outside the scope of the quantifier.

**Example:**  $\forall x \exists (y)[P(x, y, z)]$ , where **z** is a free variable.

**Bound Variable:** A variable is said to be a bound variable in a formula if it occurs within the scope of the quantifier.

**Example:**  $\forall x [A(x) B(y)]$ , here **x** and **y** are the bound variables.

# Connections between All and Exists

We can relate sentences involving  $\forall$  and  $\exists$  using De Morgan's laws:

$$(\forall x) \neg P(x) \leftrightarrow \neg(\exists x) P(x)$$

$$\neg(\forall x) P \leftrightarrow (\exists x) \neg P(x)$$

$$(\forall x) P(x) \leftrightarrow \neg (\exists x) \neg P(x)$$

$$(\exists x) P(x) \leftrightarrow \neg(\forall x) \neg P(x)$$

# Quantified inference rules

- Universal instantiation
  - $\forall x P(x) \therefore P(A)$  (A is ground term here. i.e. a term without variables)
- Universal generalization
  - $P(A) \wedge P(B) \dots \therefore \forall x P(x)$
- Existential instantiation
  - $\exists x P(x) \therefore P(F)$  ← skolem constant F
- Existential generalization
  - $P(A) \therefore \exists x P(x)$



# Universal instantiation (a.k.a. universal elimination)

- If  $(\forall x) P(x)$  is true, then  $P(C)$  is true, where  $C$  is *any* constant in the domain of  $x$
- Example:  
 $(\forall x) \text{ eats}(\text{Ziggy}, x) \Rightarrow \text{eats}(\text{Ziggy}, \text{IceCream})$
- The variable symbol can be replaced by any ground term, i.e., any constant symbol or function symbol applied to ground terms only

# Existential instantiation (a.k.a. existential elimination)

- From  $(\exists x) P(x)$  infer  $P(c)$
- Example:
  - $(\exists x) \text{ eats}(\text{Ziggy}, x) \rightarrow \text{eats}(\text{Ziggy}, \text{Stuff})$
- Note that the variable is replaced by a **brand-new constant** not occurring in this or any other sentence in the KB
- Also known as skolemization; constant is a **skolem constant**
- In other words, we don't want to accidentally draw other inferences about it by introducing the constant
- Convenient to use this to reason about the unknown object, rather than constantly manipulating the existential quantifier

# Existential generalization (a.k.a. existential introduction)

- If  $P(c)$  is true, then  $(\exists x) P(x)$  is inferred.
- Example  
 $\text{eats}(\text{Ziggy}, \text{IceCream}) \Rightarrow (\exists x) \text{eats}(\text{Ziggy}, x)$
- All instances of the given constant symbol are replaced by the new variable symbol
- Note that the variable symbol cannot already exist anywhere in the expression

# A Predicate Logic Example

1. Marcus was a man.
2. Marcus was a Pompeian.
3. All Pompeians were Romans.
4. Caesar was a ruler.
5. All Romans were either loyal to Caesar or hated him.
6. Everyone is loyal to someone.
7. People only try to assassinate rulers they are not loyal to.
8. Marcus tried to assassinate Caesar.

## A Predicate Logic Example

1. Marcus was a man.

$man(Marcus)$

2. Marcus was a Pompeian.

$Pompeian(Marcus)$

3. All Pompeians were Romans.

$\forall x : Pompeian(x) \rightarrow Roman(x)$

4. Caesar was a ruler.

$ruler(Caesar)$

5. All Romans were either loyal to Caesar or hated him.

$\forall x : Roman(x) \rightarrow loyalto(x, Caesar) \vee hate(x, Caesar)$

6. Everyone is loyal to someone.

$\forall x : \exists y : loyalto(x, y)$

7. People only try to assassinate rulers they aren't loyal to.

$\forall x : \forall y : person(x) \wedge ruler(y) \wedge tryassassinate(x, y) \rightarrow \neg loyalto(x, y)$

8. Marcus tried to assassinate Caesar.

$tryassassinate(Marcus, Caesar)$

9. All men are people.

$\forall x : man(x) \rightarrow person(x)$

# Three Ways of Representing Class Membership

1.  $man(Marcus)$
  2.  $Pompeian(Marcus)$
  3.  $\forall x : Pompeian(x) \rightarrow Roman(x)$
  4.  $ruler(Caesar)$
  5.  $\forall x : Roman(x) \rightarrow loyalto(x, Caesar) \vee hate(x, Caesar)$
- 
1.  $instance(Marcus, man)$
  2.  $instance(Marcus, Pompeian)$
  3.  $\forall x : instance(x, Pompeian) \rightarrow instance(x, Roman)$
  4.  $instance(Caesar, ruler)$
  5.  $\forall x : instance(x, Roman) \rightarrow loyalto(x, Caesar) \vee hate(x, Caesar)$
- 
1.  $instance(Marcus, man)$
  2.  $instance(Marcus, Pompeian)$
  3.  $isa(Pompeian, Roman)$
  4.  $instance(Caesar, ruler)$
  5.  $\forall x : instance(x, Roman) \rightarrow loyalto(x, Caesar) \vee hate(x, Caesar)$
  6.  $\forall x : \forall y : \forall z : instance(x, y) \wedge isa(y, z) \rightarrow instance(x, z)$

## Another Predicate Logic Example

1. Marcus was a man.
2. Marcus was a Pompeian.
3. Marcus was born in 40 A.D.
4. All men are mortal.
5. All Pompeians died when the volcano erupted in 79 A.D.
6. No mortal lives longer than 150 years.
7. It is now 1991.
8. Alive means not dead.
9. If someone dies, then he is dead at all later times.

## A Set of Facts about Marcus

1.  $man(Marcus)$
2.  $Pompeian(Marcus)$
3.  $born(Marcus, 40)$
4.  $\forall x : man(x) \rightarrow mortal(x)$
5.  $\forall x : Pompeian(x) \rightarrow died(x, 79)$
6.  $erupted(volcano, 79)$
7.  $\forall x : \forall t_1 : \forall t_2 : mortal(x) \wedge born(x, t_1) \wedge gt(t_2 - t_1, 150) \rightarrow dead(x, t_2)$
8.  $now = 1991$
9.  $\forall x : \forall t : [alive(x, t) \rightarrow \neg dead(x, t)] \wedge [\neg dead(x, t) \rightarrow alive(x, t)]$
10.  $\forall x : \forall t_1 : \forall t_2 : died(x, t_1) \wedge gt(t_2, t_1) \rightarrow dead(x, t_2)$



# Translating English to FOL

**Every gardener likes the sun.**

$$\forall x \text{ gardener}(x) \rightarrow \text{likes}(x, \text{Sun})$$

**You can fool some of the people all of the time.**

$$\exists x \forall t \text{ person}(x) \wedge \text{time}(t) \rightarrow \text{can-fool}(x, t)$$

**You can fool all of the people some of the time.**

$$\forall x \exists t (\text{person}(x) \rightarrow \text{time}(t) \wedge \text{can-fool}(x, t))$$

$$\forall x (\text{person}(x) \rightarrow \exists t (\text{time}(t) \wedge \text{can-fool}(x, t)))$$

←  
← **Equivalent**

**All purple mushrooms are poisonous.**

$$\forall x (\text{mushroom}(x) \wedge \text{purple}(x)) \rightarrow \text{poisonous}(x)$$

**No purple mushroom is poisonous.**

$$\neg \exists x \text{ purple}(x) \wedge \text{mushroom}(x) \wedge \text{poisonous}(x)$$

$$\forall x (\text{mushroom}(x) \wedge \text{purple}(x)) \rightarrow \neg \text{poisonous}(x)$$

←  
← **Equivalent**

**There are exactly two purple mushrooms.**

$$\begin{aligned} \exists x \exists y \text{ mushroom}(x) \wedge \text{purple}(x) \wedge \text{mushroom}(y) \wedge \text{purple}(y) \wedge \neg(x=y) \wedge \forall z \\ (\text{mushroom}(z) \wedge \text{purple}(z)) \rightarrow ((x=z) \vee (y=z)) \end{aligned}$$

**Clinton is not tall.**

$$\neg \text{tall}(\text{Clinton})$$

**X is above Y iff X is on directly on top of Y or there is a pile of one or more other objects directly on top of one another starting with X and ending with Y.**

$$\forall x \forall y \text{ above}(x, y) \leftrightarrow (\text{on}(x, y) \vee \exists z (\text{on}(x, z) \wedge \text{above}(z, y)))$$

# Example knowledge base

... it is a crime for an American to sell weapons to hostile nations:

$American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Nono ... has some missiles

$\exists x Owns(Nono,x) \wedge Missile(x)$

... all of its missiles were sold to it by Colonel West

$Missile(x) \wedge Owns(Nono,x) \Rightarrow Sells(West,x,Nono)$

Missiles are weapons:

$Missile(x) \Rightarrow Weapon(x)$

An enemy of America counts as "hostile":

$Enemy(x,America) \Rightarrow Hostile(x)$

West, who is American ...

$American(West)$

The country Nono, an enemy of America ...

$Enemy(Nono,America)$

# What is Unification?

- Unification is a process of making two different logical atomic expressions identical by finding a substitution. Unification depends on the substitution process.
- It takes two literals as input and makes them identical using substitution.
- Let  $\Psi_1$  and  $\Psi_2$  be two atomic sentences and  $\sigma$  be a unifier such that,  $\Psi_1\sigma = \Psi_2\sigma$ , then it can be expressed as **UNIFY( $\Psi_1, \Psi_2$ )**.
- **Example: Find the MGU for Unify{King(x), King(John)}**
- Let  $\Psi_1 = \text{King}(X)$ ,  $\Psi_2 = \text{King}(\text{John})$ ,
- **Substitution  $\theta = \{\text{John}/x\}$**  is a unifier for these atoms and applying this substitution, and both expressions will be identical.
- The UNIFY algorithm is used for unification, which takes two atomic sentences and returns a unifier for those sentences (If any exist).
- Unification is a key component of all first-order inference algorithms.
- It returns fail if the expressions do not match with each other.
- The substitution variables are called Most General Unifier or MGU.

# Conditions for Unification

- **Following are some basic conditions for unification:**
- Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
- Number of Arguments in both expressions must be identical.
- Unification will fail if there are two similar variables present in the same expression.

For each pair of the following atomic sentences find the most general unifier (If exist).

- **1. Find the MGU of  $\{p(f(a), g(Y))$  and  $p(X, X)\}$**
- Sol:  $S_0 \Rightarrow$  Here,  $\Psi_1 = p(f(a), g(Y))$ , and  $\Psi_2 = p(X, X)$   
SUBST  $\theta = \{f(a) / X\}$   
 $S_1 \Rightarrow \Psi_1 = p(f(a), g(Y))$ , and  $\Psi_2 = p(f(a), f(a))$   
SUBST  $\theta = \{f(a) / g(Y)\}$ , **Unification failed.**

- **2. Find the MGU of  $\{p(b, X, f(g(Z)))$  and  $p(Z, f(Y), f(Y))\}$**
- Here,  $\Psi_1 = p(b, X, f(g(Z)))$  , and  $\Psi_2 = p(Z, f(Y), f(Y))$   
 $S_0 \Rightarrow \{ p(b, X, f(g(Z))); p(Z, f(Y), f(Y)) \}$   
SUBST  $\theta = \{b/Z\}$
- $S_1 \Rightarrow \{ p(b, X, f(g(b))); p(b, f(Y), f(Y)) \}$   
SUBST  $\theta = \{f(Y) / X\}$
- $S_2 \Rightarrow \{ p(b, f(Y), f(g(b))); p(b, f(Y), f(Y)) \}$   
SUBST  $\theta = \{g(b) / Y\}$
- $S_2 \Rightarrow \{ p(b, f(g(b)), f(g(b))); p(b, f(g(b)), f(g(b))) \}$  **Unified Successfully.**  
**And Unifier =  $\{ b/Z, f(Y) / X, g(b) / Y \}$ .**

- **3. Find the MGU of  $\{p(X, X), \text{ and } p(Z, f(Z))\}$**
- Here,  $\Psi_1 = \{p(X, X), \text{ and } \Psi_2 = p(Z, f(Z))\}$   
 $S_0 \Rightarrow \{p(X, X), p(Z, f(Z))\}$   
 SUBST  $\theta = \{X/Z\}$   
 $S1 \Rightarrow \{p(Z, Z), p(Z, f(Z))\}$   
 SUBST  $\theta = \{f(Z) / Z\}$ , **Unification Failed.**

# Resolution in FOL

- Resolution is a theorem proving technique that proceeds by building refutation proofs, i.e., proofs by contradictions. It was invented by a Mathematician John Alan Robinson in the year 1965.
- Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements. Unification is a key concept in proofs by resolutions. Resolution is a single inference rule which can efficiently operate on the **conjunctive normal form or clausal form**.
- **Clause**: Disjunction of literals (an atomic sentence) is called a **clause**. It is also known as a unit clause.



# Example:

- We can resolve two clauses which are given below:
- **$[\text{Animal}(g(x) \vee \text{Loves}(f(x), x)]$       and       $[\neg \text{Loves}(a, b) \vee \neg \text{Kills}(a, b)]$**
- Where two complimentary literals are:  **$\text{Loves}(f(x), x)$  and  $\neg \text{Loves}(a, b)$**
- These literals can be unified with unifier  **$\theta = [a/f(x), \text{ and } b/x]$**  , and it will generate a resolvent clause:
- **$[\text{Animal}(g(x) \vee \neg \text{Kills}(f(x), x)]$ .**

# Steps for Resolution:

- Conversion of facts into first-order logic.
- Convert FOL statements into CNF
- Negate the statement which needs to prove (proof by contradiction)
- Draw resolution graph (unification).

# Example:

- **John likes all kind of food.**
- **Apple and vegetable are food**
- **Anything anyone eats and not killed is food.**
- **Anil eats peanuts and still alive**
- **Harry eats everything that Anil eats.**

**Prove by resolution that:**

- **John likes peanuts.**

# Step-1: Conversion of Facts into FOL

- In the first step we will convert all the given statements into its first order logic.

a.  $\forall x: \text{food}(x) \rightarrow \text{likes}(\text{John}, x)$

b.  $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$

c.  $\forall x \forall y: \text{eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$

d.  $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$ .

e.  $\forall x : \text{eats}(\text{Anil}, x) \rightarrow \text{eats}(\text{Harry}, x)$

f.  $\forall x: \neg \text{killed}(x) \rightarrow \text{alive}(x)$

g.  $\forall x: \text{alive}(x) \rightarrow \neg \text{killed}(x)$

} added predicates.

h.  $\text{likes}(\text{John}, \text{Peanuts})$

# CNF

- A sentence is in Conjunctive Normal Form (CNF) if it is a conjunction of clauses, each clause being a disjunction of literals
- Examples:

$$(A \vee B) \wedge (C \vee D \vee J) \wedge (E \vee G)$$

## Conversion to CNF

$$P \leftrightarrow (Q \vee R)$$

1. Eliminate  $\leftrightarrow$ , replacing  $\alpha \leftrightarrow \beta$  with  $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$ .

$$(P \rightarrow (Q \vee R)) \wedge ((Q \vee R) \rightarrow P)$$

2. Eliminate  $\rightarrow$ , replacing  $\alpha \rightarrow \beta$  with  $\neg \alpha \vee \beta$ .

$$(\neg P \vee Q \vee R) \wedge (\neg(Q \vee R) \vee P)$$

3. Move  $\neg$  inwards using de Morgan's rules and double-negation:

$$(\neg P \vee Q \vee R) \wedge ((\neg Q \wedge \neg R) \vee P)$$

4. Apply distributivity law ( $\wedge$  over  $\vee$ ) and flatten:

$$(\neg P \vee Q \vee R) \wedge (\neg Q \vee P) \wedge (\neg R \vee P)$$

# Step-2: Conversion of FOL into CNF

- a.  $\forall x: \text{food}(x) \rightarrow \text{likes}(\text{John}, x)$
  - b.  $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
  - c.  $\forall x \forall y: \text{eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$
  - d.  $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$ .
  - e.  $\forall x: \text{eats}(\text{Anil}, x) \rightarrow \text{eats}(\text{Harry}, x)$
  - f.  $\forall x: \neg \text{killed}(x) \rightarrow \text{alive}(x)$
  - g.  $\forall x: \text{alive}(x) \rightarrow \neg \text{killed}(x)$
  - h.  $\text{likes}(\text{John}, \text{Peanuts})$
- } **added predicates.**

- **Eliminate all implication ( $\rightarrow$ ) and rewrite**
- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall x \forall y \neg [\text{eats}(x, y) \wedge \neg \text{killed}(x)] \vee \text{food}(y)$
- $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
- $\forall x \neg [\neg \text{killed}(x)] \vee \text{alive}(x)$
- $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$
- $\text{likes}(\text{John}, \text{Peanuts})$ .

# Move negation ( $\neg$ ) inwards and rewrite

- **Eliminate all implication ( $\rightarrow$ ) and rewrite**

- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall x \forall y \neg [\text{eats}(x, y) \wedge \neg \text{killed}(x)] \vee \text{food}(y)$
- $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
- $\forall x \neg [\neg \text{killed}(x)] \vee \text{alive}(x)$
- $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$
- $\text{likes}(\text{John}, \text{Peanuts})$ .

- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall x \forall y \neg \text{eats}(x, y) \vee \text{killed}(x) \vee \text{food}(y)$
- $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
- $\forall x \text{killed}(x) \vee \text{alive}(x)$
- $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$
- $\text{likes}(\text{John}, \text{Peanuts})$ .



# Rename variables or standardize variables

- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall y \forall z \neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$
- $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall w \neg \text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w)$
- $\forall g \neg \text{killed}(g) \vee \text{alive}(g)$
- $\forall k \neg \text{alive}(k) \vee \neg \text{killed}(k)$
- $\text{likes}(\text{John}, \text{Peanuts})$ .

- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall y \forall z \neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$
- $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall w \neg \text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w)$
- $\forall g \text{killed}(g) \vee \text{alive}(g)$
- $\forall k \neg \text{alive}(k) \vee \neg \text{killed}(k)$
- $\text{likes}(\text{John}, \text{Peanuts})$ .

## • Eliminate existential instantiation quantifier by elimination.

- In this step, we will eliminate existential quantifier  $\exists$ , and this process is known as **Skolemization**. But in this example problem since there is no existential quantifier so all the statements will remain same in this step.

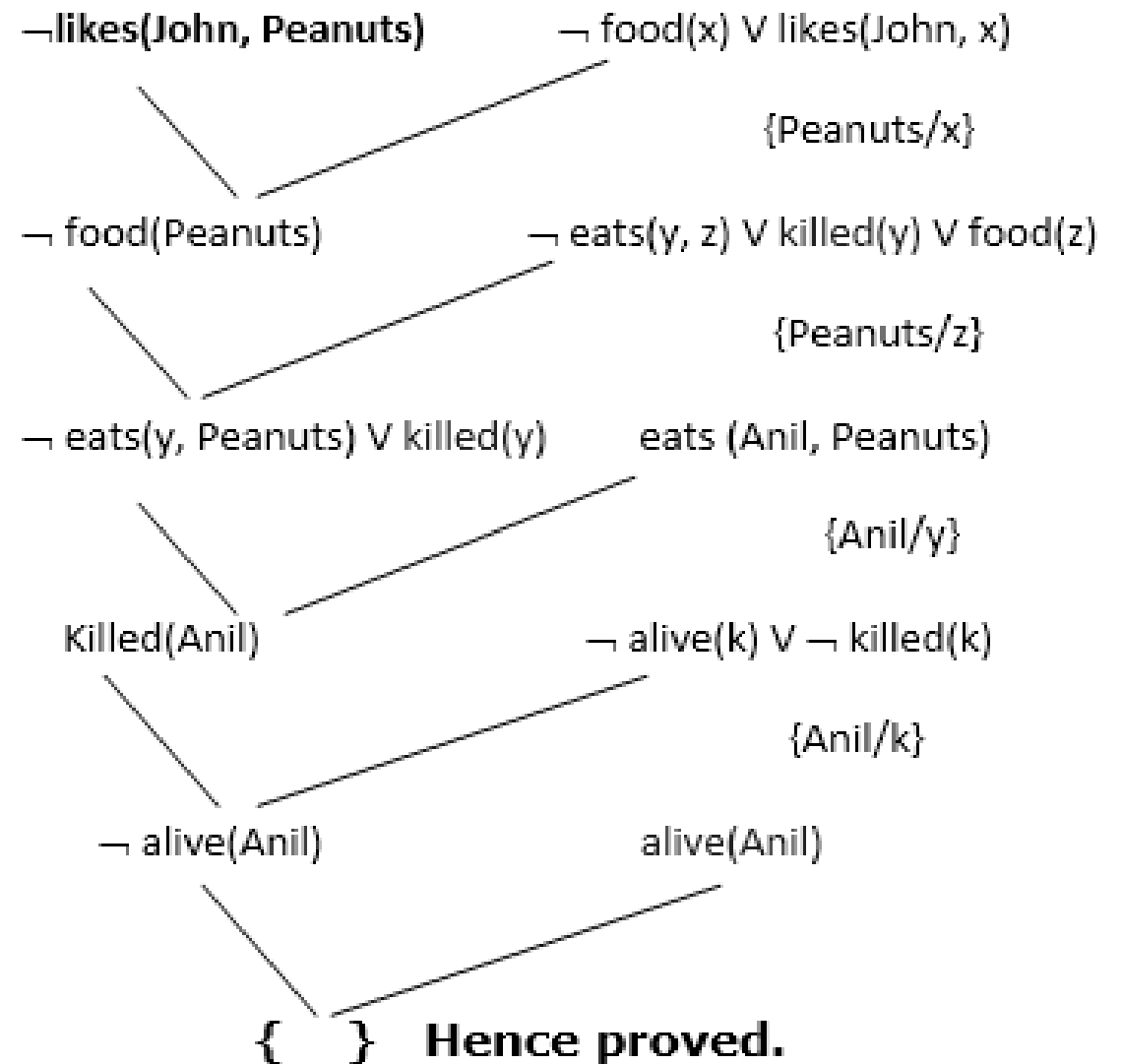
## • Drop Universal quantifiers.

- In this step we will drop all universal quantifier since all the statements are not implicitly quantified so we don't need it.
- $\neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple})$
- $\text{food}(\text{vegetables})$
- $\neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$
- $\text{eats}(\text{Anil}, \text{Peanuts})$
- $\text{alive}(\text{Anil})$
- $\neg \text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w)$
- $\text{killed}(g) \vee \text{alive}(g)$
- $\neg \text{alive}(k) \vee \neg \text{killed}(k)$
- $\text{likes}(\text{John}, \text{Peanuts})$ .

- **Distribute conjunction  $\wedge$  over disjunction  $\vee$ .**  
This step will not make any change in this problem.
- **Step-3: Negate the statement to be proved**
- In this statement, we will apply negation to the conclusion statements, which will be written as  $\neg \text{likes}(\text{John}, \text{Peanuts})$

## Step-4: Draw Resolution graph:

- $\neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple})$
- $\text{food}(\text{vegetables})$
- $\neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$
- $\text{eats}(\text{Anil}, \text{Peanuts})$
- $\text{alive}(\text{Anil})$
- $\neg \text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w)$
- $\text{killed}(g) \vee \text{alive}(g)$
- $\neg \text{alive}(k) \vee \neg \text{killed}(k)$
- $\text{likes}(\text{John}, \text{Peanuts})$ .



## Using Resolution in First order logic Prove the given Sentence

- The law states that, it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West who is American.
- Prove that West is a criminal

- it is a crime for an American to sell weapons to hostile nations
- Country Nono has some missiles.
- all of its missiles were sold to it by Colonel West
- Missiles are weapons
- An enemy of America counts as "hostile"
- West, who is American
- The country Nono, an enemy of America

# Example knowledge base

... it is a crime for an American to sell weapons to hostile nations:

$American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Nono ... has some missiles, i.e.,  $\exists x Owns(Nono,x) \wedge Missile(x)$ : convert to two definite clauses

$Owns(Nono,M_1)$  and  $Missile(M_1)$

... all of its missiles were sold to it by Colonel West

$Missile(x) \wedge Owns(Nono,x) \Rightarrow Sells(West,x,Nono)$

Missiles are weapons:

$Missile(x) \Rightarrow Weapon(x)$

An enemy of America counts as "hostile":

$Enemy(x,America) \Rightarrow Hostile(x)$

West, who is American ...

$American(West)$

The country Nono, an enemy of America ...

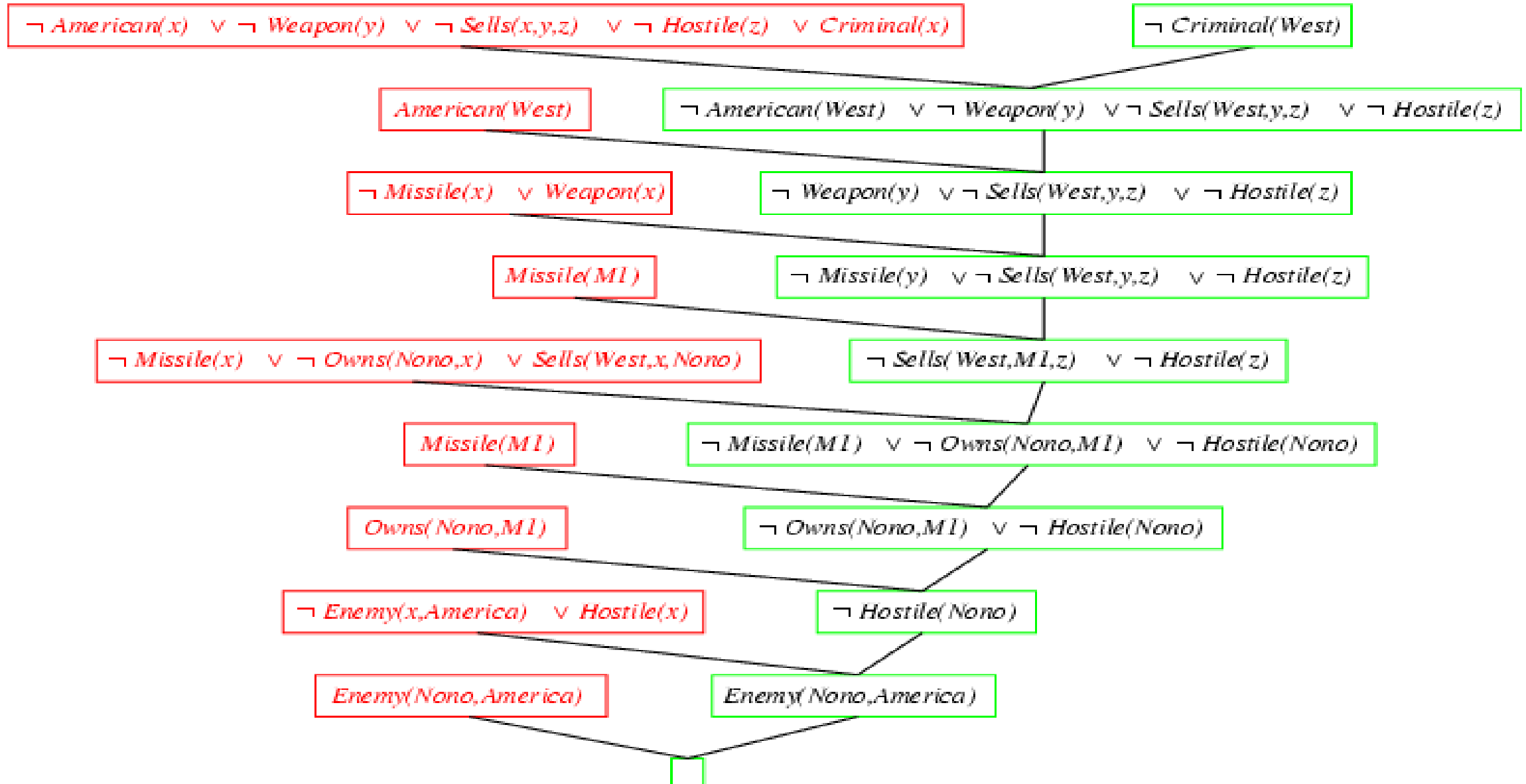
$Enemy(Nono,America)$

# CNF

$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$   
 $\neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{West}, x, \text{Nono}) .$   
 $\neg \text{Enemy}(x, \text{America}) \vee \text{Hostile}(x) .$   
 $\neg \text{Missile}(x) \vee \text{Weapon}(x) .$   
 $\text{Owns}(\text{Nono}, M_1) . \quad \text{Missile}(M_1) .$   
 $\text{American}(\text{West}) \quad \text{Enemy}(\text{Nono}, \text{America}) .$



# Resolution proof: definite clauses



# Resolution example

Everyone who loves all animals is loved by someone.

Anyone who kills an animal is loved by no one.

Jack loves all animals.

Either Jack or Curiosity killed the cat, who is named Tuna.

Did Curiosity kill the cat?

# Conversion to CNF

- Everyone who loves all animals is loved by someone:

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x,y)] \Rightarrow [\exists y \text{ Loves}(y,x)]$$

- 1. Eliminate biconditionals and implications

- 

$$\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

- 2. Move  $\neg$  inwards:  $\neg \forall x p \equiv \exists x \neg p$ ,  $\neg \exists x p \equiv \forall x \neg p$

$$\forall x [\exists y \neg(\neg \text{Animal}(y) \vee \text{Loves}(x,y))] \vee [\exists y \text{ Loves}(y,x)]$$

$$\forall x [\exists y \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

$$\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

# Conversion to CNF contd.

3. Standardize variables: each quantifier should use a different one

$$\forall x [\exists y \textit{Animal}(y) \wedge \neg \textit{Loves}(x,y)] \vee [\exists z \textit{Loves}(z,x)]$$

4. Skolemize: a more general form of existential instantiation.

Each existential variable is replaced by a **Skolem function** of the enclosing universally quantified variables:

$$\forall x [\textit{Animal}(F(x)) \wedge \neg \textit{Loves}(x, F(x))] \vee \textit{Loves}(G(x),x)$$

5. Drop universal quantifiers:

$$[\textit{Animal}(F(x)) \wedge \neg \textit{Loves}(x,F(x))] \vee \textit{Loves}(G(x),x)$$

6. Distribute  $\vee$  over  $\wedge$  :

$$[\textit{Animal}(F(x)) \vee \textit{Loves}(G(x),x)] \wedge [\neg \textit{Loves}(x,F(x)) \vee \textit{Loves}(G(x),x)]$$

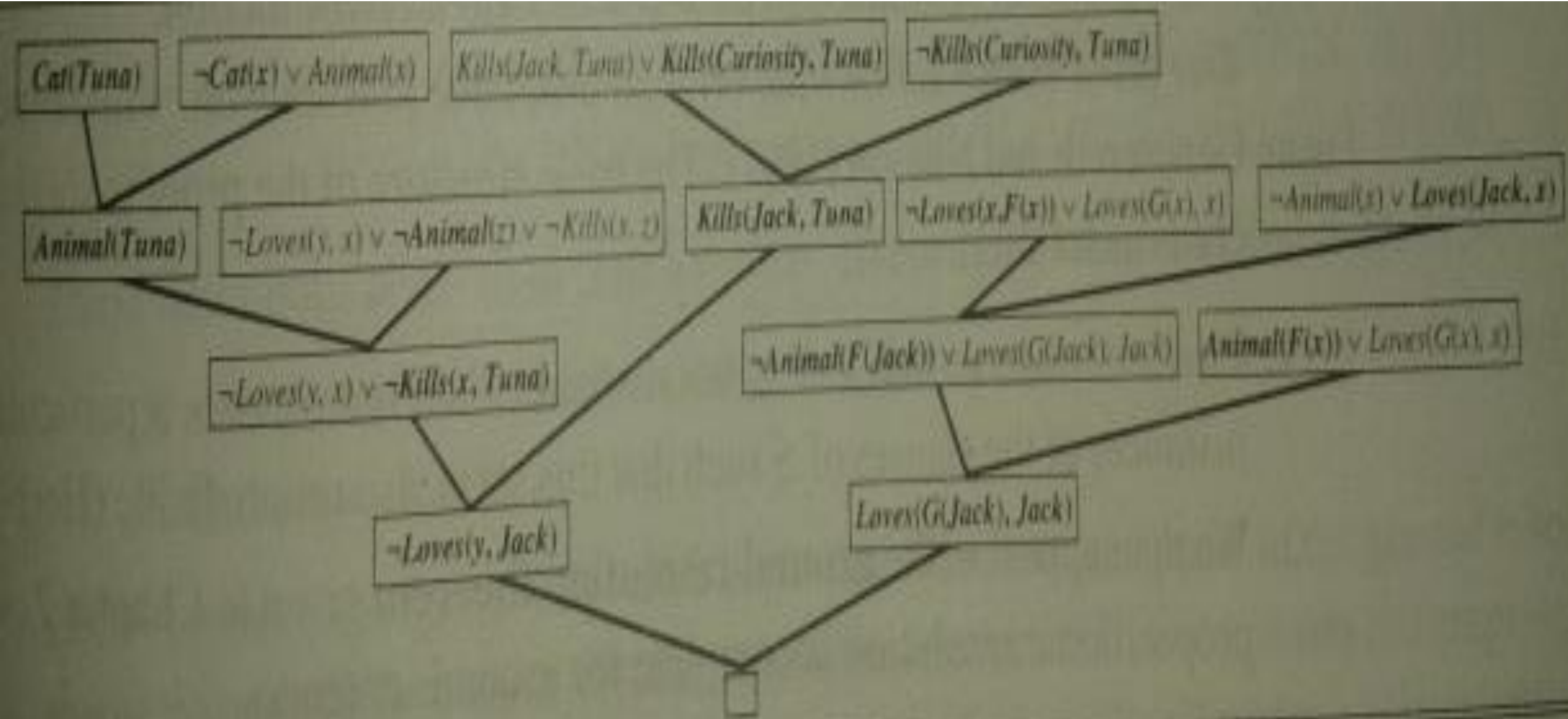
# FOL

- A.  $\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)]$
- B.  $\forall x [\exists y \text{ Animal}(y) \wedge \text{Kills}(x, y)] \Rightarrow [\forall z \neg \text{Loves}(z, x)]$
- C.  $\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack}, x)$
- D.  $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$
- E.  $\text{Cat}(\text{Tuna})$
- F.  $\forall x \text{ Cat}(x) \Rightarrow \text{Animal}(x)$
- G.  $\neg \text{Kills}(\text{Curiosity}, \text{Tuna})$

# CNF

- A1.  $Animal(F(x)) \vee Loves(G(x), x)$
- A2.  $\neg Loves(x, F(x)) \vee Loves(G(x), x)$
- B.  $\neg Animal(y) \vee \neg Kills(x, y) \vee \neg Loves(z, x)$
- C.  $\neg Animal(x) \vee Loves(Jack, x)$
- D.  $Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$
- E.  $Cat(Tuna)$
- F.  $\neg Cat(x) \vee Animal(x)$
- $\neg$ G.  $\neg Kills(Curiosity, Tuna)$

# Resolution Proof (who kill the cat?)



# Forward chaining

- The inference engine is the component of the intelligent system in artificial intelligence, which applies logical rules to the knowledge base to infer new information from known facts. The first inference engine was part of the expert system. Inference engine commonly proceeds in two modes, which are:
  - **Forward chaining**
  - **Backward chaining**



- Horn clause and definite clause are the forms of sentences, which enables knowledge base to use a more restricted and efficient inference algorithm. Logical inference algorithms use forward and backward chaining approaches, which require KB in the form of the **first-order definite clause**.
- **Definite clause:** A clause which is a disjunction of literals with **exactly one positive literal** is known as a definite clause or strict horn clause.
- **Horn clause:** A clause which is a disjunction of literals with **at most one positive literal** is known as horn clause. Hence all the definite clauses are horn clauses.
- **Example:**  $(\neg p \vee \neg q \vee k)$ . It has only one positive literal  $k$ .
- It is equivalent to  $p \wedge q \rightarrow k$ .

- Forward chaining is also known as a forward deduction or forward reasoning method when using an inference engine. Forward chaining is a form of reasoning which start with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached.
- The Forward-chaining algorithm starts from known facts, triggers all rules whose premises are satisfied, and add their conclusion to the known facts. This process repeats until the problem is solved.

- **Properties of Forward-Chaining:**

- It is a down-up approach, as it moves from bottom to top.
- It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
- Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.
-

# Example knowledge base

- The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.
- 
- Prove that Col. West is a criminal
-

# Example knowledge base

... it is a crime for an American to sell weapons to hostile nations:

$American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Nono ... has some missiles, i.e.,  $\exists x Owns(Nono,x) \wedge Missile(x)$ : convert to two definite clauses

$Owns(Nono,M_1)$  and  $Missile(M_1)$

... all of its missiles were sold to it by Colonel West

$Missile(x) \wedge Owns(Nono,x) \Rightarrow Sells(West,x,Nono)$

Missiles are weapons:

$Missile(x) \Rightarrow Weapon(x)$

An enemy of America counts as "hostile":

$Enemy(x,America) \Rightarrow Hostile(x)$

West, who is American ...

$American(West)$

The country Nono, an enemy of America ...

$Enemy(Nono,America)$

# Forward chaining proof

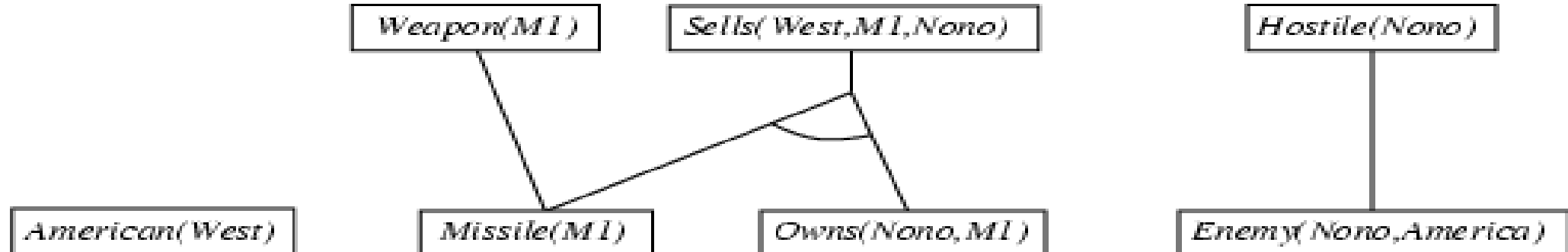
*American(West)*

*Missile(MI)*

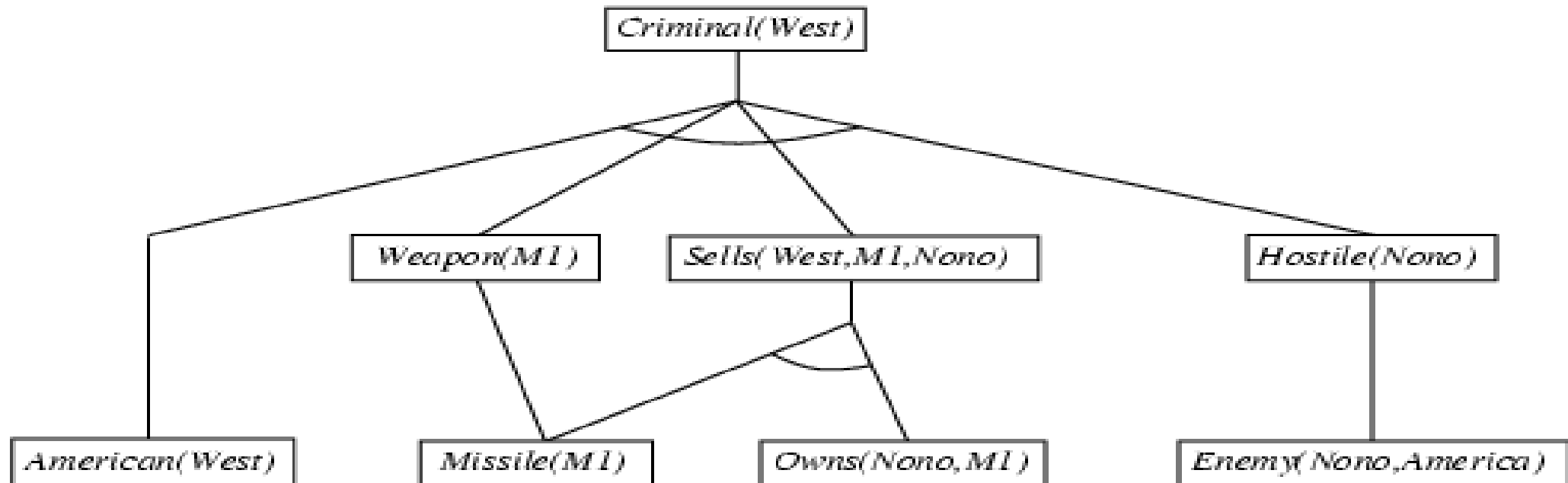
*Owns(Nono,MI)*

*Enemy(Nono,America)*

# Forward chaining proof



# Forward chaining proof





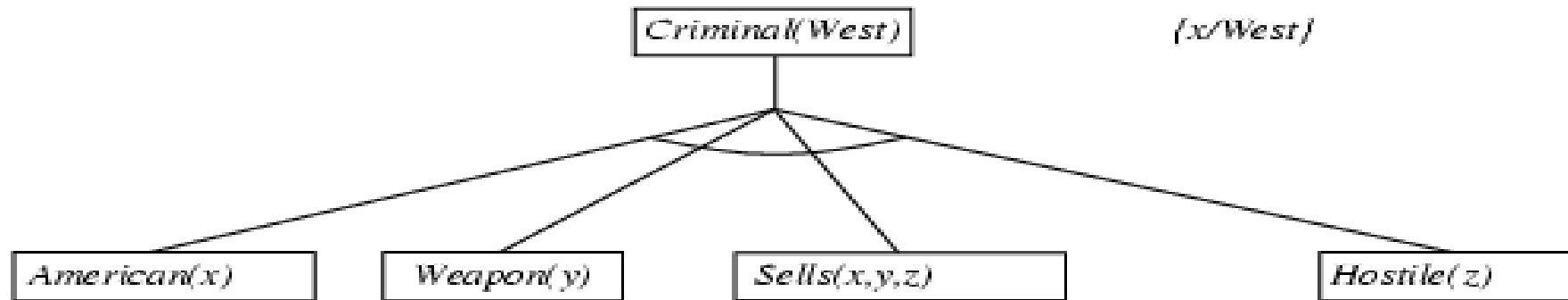
# Backward Chaining:

- Backward-chaining is also known as a backward deduction or backward reasoning method when using an inference engine. A backward chaining algorithm is a form of reasoning, which starts with the goal and works backward, chaining through rules to find known facts that support the goal.
- **Properties of backward chaining:**
- It is known as a top-down approach.
- Backward-chaining is based on modus ponens inference rule.
- In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.
- It is called a goal-driven approach, as a list of goals decides which rules are selected and used.
- Backward -chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.
- The backward-chaining method mostly used a **depth-first search** strategy for proof.
-

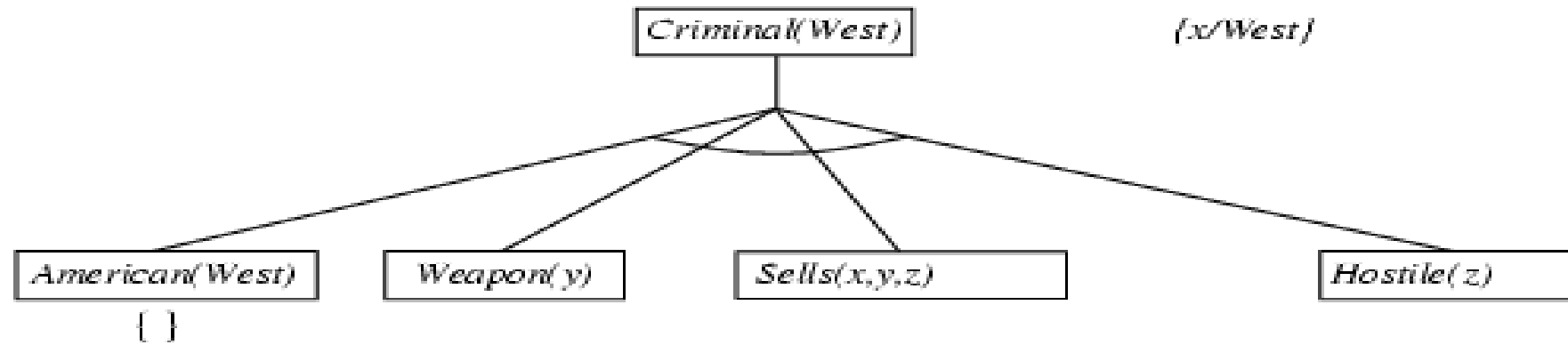
# Backward chaining example

*Criminal(West)*

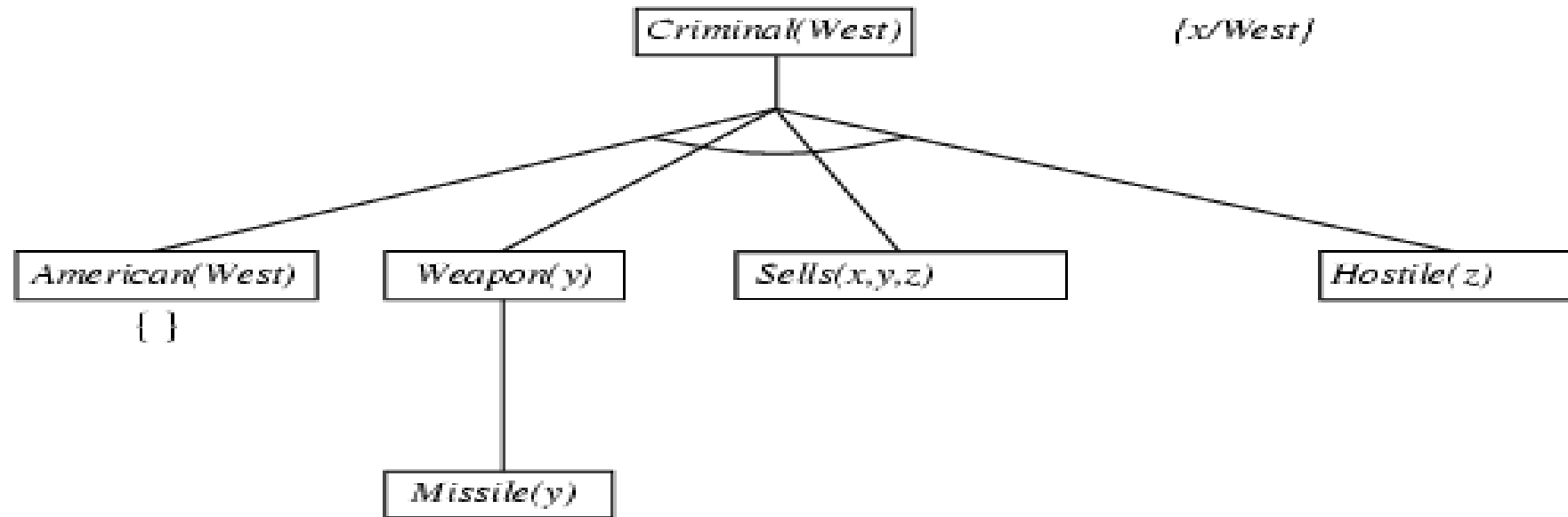
# Backward chaining example



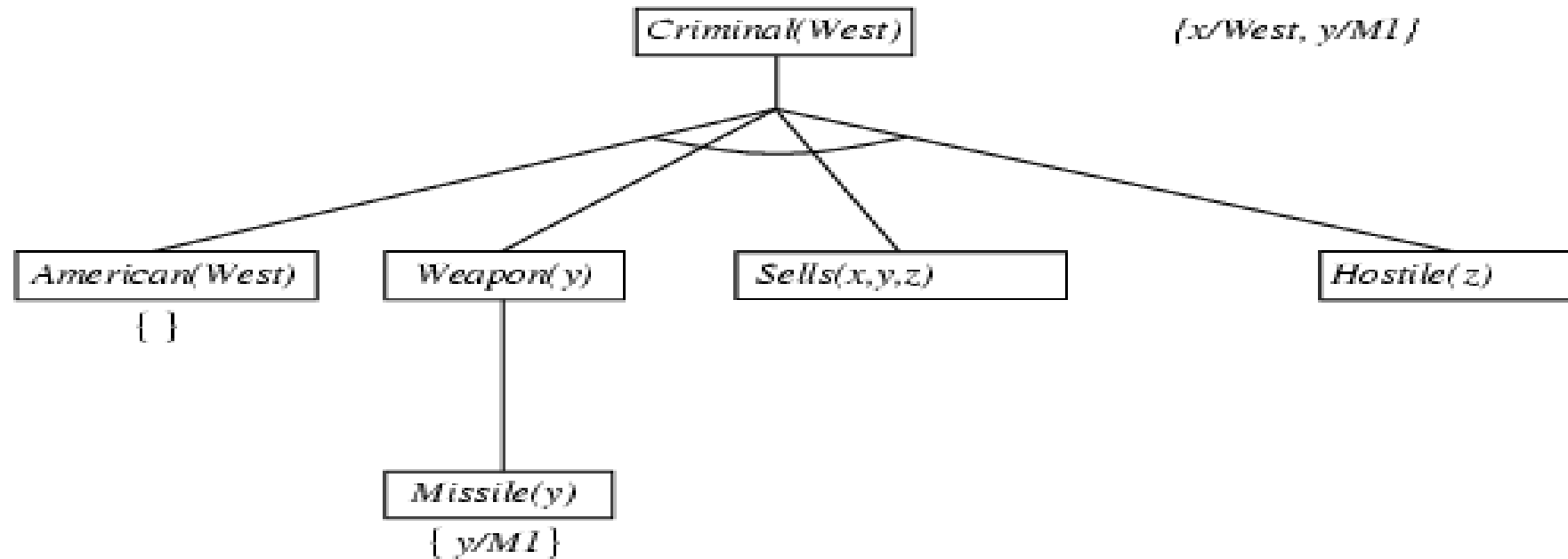
# Backward chaining example



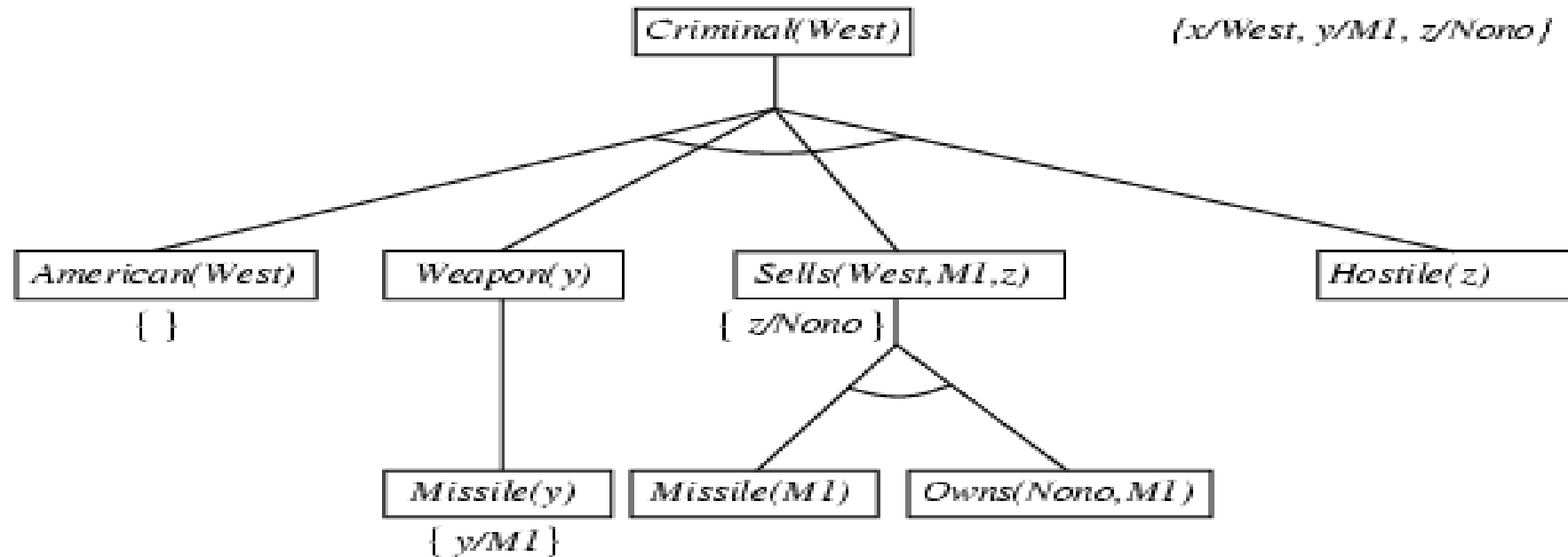
# Backward chaining example



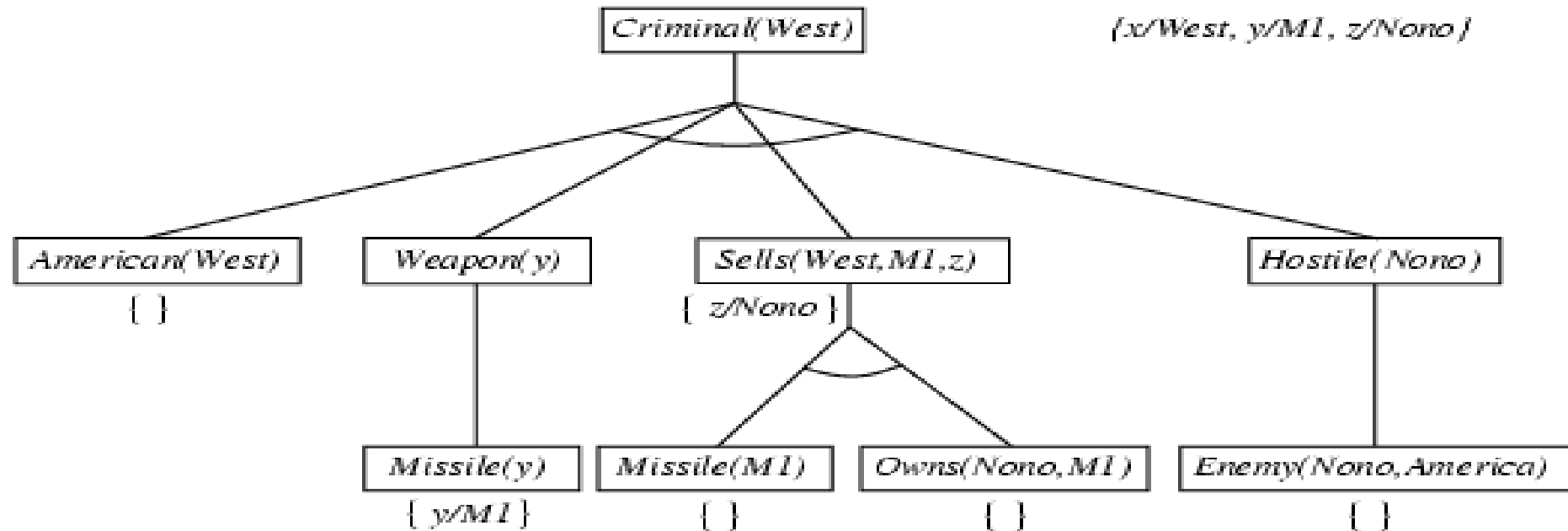
# Backward chaining example



# Backward chaining example

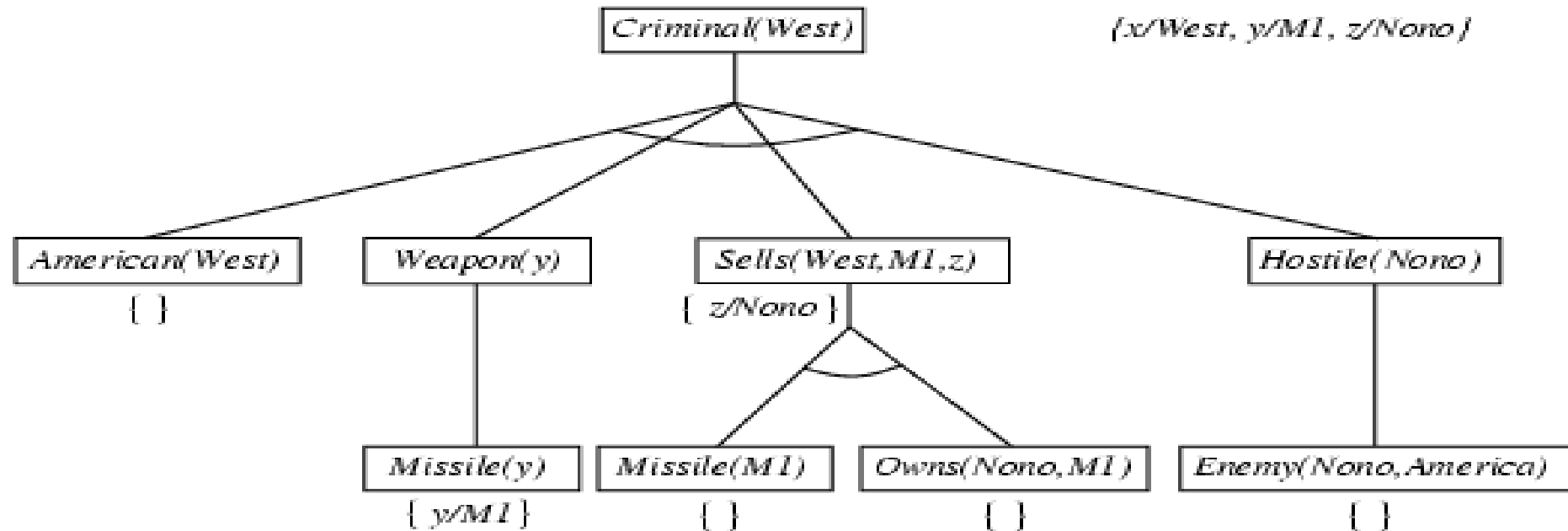


# Backward chaining example





# Backward chaining example



| S. No. | Forward Chaining                                                                                                      | Backward Chaining                                                                                                                   |
|--------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 1.     | Forward chaining starts from known facts and applies inference rule to extract more data unit it reaches to the goal. | Backward chaining starts from the goal and works backward through inference rules to find the required facts that support the goal. |
| 2.     | It is a bottom-up approach                                                                                            | It is a top-down approach                                                                                                           |
| 3.     | Forward chaining is known as data-driven inference technique as we reach to the goal using the available data.        | Backward chaining is known as goal-driven technique as we start from the goal and divide into sub-goal to extract the facts.        |
| 4.     | Forward chaining reasoning applies a breadth-first search strategy.                                                   | Backward chaining reasoning applies a depth-first search strategy.                                                                  |
| 5.     | Forward chaining tests for all the available rules                                                                    | Backward chaining only tests for few required rules.                                                                                |
| 6.     | Forward chaining is suitable for the planning, monitoring, control, and interpretation application.                   | Backward chaining is suitable for diagnostic, prescription, and debugging application.                                              |
| 7.     | Forward chaining can generate an infinite number of possible conclusions.                                             | Backward chaining generates a finite number of possible conclusions.                                                                |
| 8.     | It operates in the forward direction.                                                                                 | It operates in the backward direction.                                                                                              |
| 9.     | Forward chaining is aimed for any conclusion.                                                                         | Backward chaining is only aimed for the required data.                                                                              |



